

Experiment 'p_6_9_arrays_equal_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_9_arrays_equal_array

Natural language statement: null

Method signature: p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal: bool)

Ensures

- are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.Length ==> first[i] == second[i])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal: bool)`

The method should respect the following contract:

`ensures are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.Length
 ↪ ==> first[i] == second[i])`

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnORNw0YgDec9jsrUmeSaSsp726Qq

Model: gpt-5.5-2026-04-23

Created at: 1780664393

Finish reason: stop

Usage: 304 tokens in, and 4470 tokens out

```
{  
  if first.Length != second.Length {  
    are_equal := false;  
    return;  
  }  
  
  var i := 0;  
  while i < first.Length  
    invariant 0 <= i <= first.Length  
    invariant first.Length == second.Length  
    invariant forall j :: 0 <= j < i ==> first[j] == second[j]  
  {  
    if first[i] != second[i] {  
      are_equal := false;  
      assert !(forall j :: 0 <= j < first.Length ==> first[j] == second[j]) by {  
        if forall j :: 0 <= j < first.Length ==> first[j] == second[j] {  
          assert first[i] == second[i];  
          assert false;  
        }  
      }  
    }  
  }  
}
```

```

    return;
}

assert first[i] == second[i];
assert forall j | 0 <= j < i + 1 :: first[j] == second[j] by {
  if j < i {
    assert first[j] == second[j];
  } else {
    assert j == i;
  }
}
i := i + 1;
}

assert i == first.Length;
assert forall j | 0 <= j < first.Length :: first[j] == second[j] by {
  assert 0 <= j < i;
  assert first[j] == second[j];
}
are_equal := true;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_6_9_arrays_equal_array/program.dfy(28,9): Error: unresolved identifier: j
|
28 | if j < i {
| ^

problem_data/p_6_9_arrays_equal_array/program.dfy(29,21): Error: unresolved identifier: j
|
29 | assert first[j] == second[j];
| ^

problem_data/p_6_9_arrays_equal_array/program.dfy(29,34): Error: unresolved identifier: j
|
29 | assert first[j] == second[j];
| ^

problem_data/p_6_9_arrays_equal_array/program.dfy(31,15): Error: unresolved identifier: j
|
31 | assert j == i;
| ^

problem_data/p_6_9_arrays_equal_array/program.dfy(39,16): Error: unresolved identifier: j
|
39 | assert 0 <= j < i;
| ^

```

```

problem_data/p_6_9_arrays_equal_array/program.dfy(40,17): Error: unresolved identifier: j
|
40 | assert first[j] == second[j];
| ^

problem_data/p_6_9_arrays_equal_array/program.dfy(40,30): Error: unresolved identifier: j
|
40 | assert first[j] == second[j];
| ^

7 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOSucvFpOdcSPDMraX44F4TqW3i7

Model: gpt-5.5-2026-04-23

Created at: 1780664488

Finish reason: stop

Usage: 1260 tokens in, and 1981 tokens out

```

{
  if first.Length != second.Length {
    are_equal := false;
    assert first.Length != second.Length;
    return;
  }

  var i := 0;
  while i < first.Length
    invariant 0 <= i <= first.Length
    invariant first.Length == second.Length
    invariant forall j :: 0 <= j < i ==> first[j] == second[j]
  {
    if first[i] != second[i] {
      are_equal := false;
      assert 0 <= i < first.Length;
      assert first[i] != second[i];
      assert !(forall j :: 0 <= j < first.Length ==> first[j] == second[j]) by {
        if forall j :: 0 <= j < first.Length ==> first[j] == second[j] {
          assert first[i] == second[i];
          assert false;
        }
      }
    }
    return;
  }
}

```

```

    assert first[i] == second[i];
    forall j | 0 <= j < i + 1
      ensures first[j] == second[j]
    {
      if j < i {
        assert first[j] == second[j];
      } else {
        assert j == i;
        assert first[j] == second[j];
      }
    }
  }
  assert forall j :: 0 <= j < i + 1 ==> first[j] == second[j];
  i := i + 1;
}

assert i == first.Length;
forall j | 0 <= j < first.Length
  ensures first[j] == second[j]
{
  assert 0 <= j < i;
  assert first[j] == second[j];
}
assert forall j :: 0 <= j < first.Length ==> first[j] == second[j];
are_equal := true;
}

```

Final Program

```

method p_6_9_arrays_equal_array(first: array<int>, second: array<int>) returns (are_equal
  ↪ : bool)
  ensures are_equal == (first.Length == second.Length && forall i :: 0 <= i < first.
    ↪ Length ==> first[i] == second[i])
{
  if first.Length != second.Length {
    are_equal := false;
    assert first.Length != second.Length;
    return;
  }

  var i := 0;
  while i < first.Length
    invariant 0 <= i <= first.Length
    invariant first.Length == second.Length
    invariant forall j :: 0 <= j < i ==> first[j] == second[j]
  {
    if first[i] != second[i] {
      are_equal := false;
      assert 0 <= i < first.Length;
      assert first[i] != second[i];
      assert !(forall j :: 0 <= j < first.Length ==> first[j] == second[j]) by {

```

```

        if forall j :: 0 <= j < first.Length ==> first[j] == second[j] {
            assert first[i] == second[i];
            assert false;
        }
    }
    return;
}

assert first[i] == second[i];
forall j | 0 <= j < i + 1
    ensures first[j] == second[j]
{
    if j < i {
        assert first[j] == second[j];
    } else {
        assert j == i;
        assert first[j] == second[j];
    }
}
assert forall j :: 0 <= j < i + 1 ==> first[j] == second[j];
i := i + 1;
}

assert i == first.Length;
forall j | 0 <= j < first.Length
    ensures first[j] == second[j]
{
    assert 0 <= j < i;
    assert first[j] == second[j];
}
assert forall j :: 0 <= j < first.Length ==> first[j] == second[j];
are_equal := true;
}

```

Total Token Usage

Input tokens: 1564

Output tokens: 6451

Reasoning tokens: 5624

Sum of 'total tokens': 8015

Experiment Timings

Overall Experiment started at 1780664393510, ended at 1780664556607, lasting 163097ms (163.10 seconds)

Iteration #1 started at 1780664393511, ended at 1780664488304, lasting 94793ms (94.79 seconds)

Iteration #2 started at 1780664488304, ended at 1780664556607, lasting 68303ms (68.30 seconds)